UNIVERSITÀ DEGLI STUDI DI PISA

Decentralised Lending Service

PEER TO PEER SYSTEMS AND BLOCKCHAINS

Project

Student
Simone Infantino
723462

Student
Antonio Tamborrino
564647

Academic Year
2025/2026

Contents

Chapter 1

Intro and Implementation Choices

1.1 Smart Contract Implementation

The decentralized loan platform is implemented in Solidity and developed using the Hardhat framework with the Viem interaction layer. Its architecture consists of three smart contracts: `LoanService`, which manages the protocol; `Loan`, which represents individual loans; and `BitcoinOracle`, which provides Bitcoin collateral information. This separation is natural as these contracts represent the actors of the project.

1.2 Loan Service Architecture

The `LoanService` contract manages liquidity, loan proposals, voting, contributor balances, and oracle interaction. Rather than storing all loans internally, each approved proposal deploys an independent `Loan` contract containing its own state.
Contributor funds are divided into available deposits and locked liquidity, ensuring that only uncommitted funds can be withdrawn. A minimum deposit is required to participate, and inactive contributors are removed from the registry to reduce storage costs.

1.3 Loan Proposal and Voting Mechanism

Applicants submit loan proposals specifying the requested amount, interest rate, duration, and Bitcoin collateral address. Proposals remain open for a fixed voting period during which contributors vote with a weight proportional to their available liquidity.
A proposal is approved only if sufficient liquidity is available, the Bitcoin balance provided by the oracle satisfies the collateral requirement, and approval votes exceed rejection votes. Successful proposals generate a new `Loan` contract and lock contributor funds proportionally.

1.4 Loan Contract Design

Each `Loan` contract represents a single loan agreement and transfers the requested funds to the borrower immediately after deployment.
The contract stores immutable loan parameters, including the borrower, lent amount, interest rate, duration, collateral ratio, and Bitcoin address. Repayments can be partial or complete and are distributed proportionally between base, interest and collateral (dictated by the collateral percentage).
The contract also stores the contributors participating in the loan together with their locked amounts, allowing repayments to be distributed proportionally.

1.5 Bitcoin Oracle Integration

The `BitcoinOracle` contract supplies Bitcoin balance information required to validate the Bitcoin address balance provided at loan creation. Applicants may request an update by paying a minimum fee, while an off-chain oracle service scans the Bitcoin blockchain and the oracle owner submits the updated balances on-chain.
This design separates on-chain logic from external data acquisition, with only the oracle owner authorized to update stored balances. The immutable deployment block is also stored to allow the off-chain service to resume synchronization from the correct point.

1.6 Security and Administrative Choices

The contracts adopt a two-step ownership transfer mechanism for both the loan service and the oracle to prevent accidental administrative changes. Both contracts also support controlled termination, while the loan service additionally enables migration to a successor contract once all active loans have been completed.
The loan service also supports controlled protocol migration by allowing termination only after all active loans are completed. During migration, the remaining Ether balance is transferred to a designated successor contract. Callback functions enable communication between `Loan` and `LoanService` while preserving modularity.
Finally, the project includes intentionally vulnerable contracts and a reentrancy attacker used to evaluate security measures and validate protection against reentrancy attacks.

Chapter 2

Reentrancy and Security Discussion

2.1 Gas Evaluation

TABLE 2.1: BitcoinOracle Gas consumption.

Function name	Min	Average	Median	Max
push_balance	31616	49857	51516	51516
transfer_ownership	47792	47792	47792	47792
terminate	46264	46264	46264	46264
request_update	29352	29352	29352	29352
accept_ownership	28218	28218	28218	28218
get_balance	24845	24845	24845	24845
Deployment	679532	679532	679532	679532

`push_balance` ranges from 31000 to 51000 gas. The spread is purely storage pricing, since recording a Bitcoin address the oracle has never seen allocates a fresh slot (20000 gas). `get_balance` reports 24845 gas, but 21000 of that is the base transaction fee. This difference, shown also in the other two contracts' gas evaluations, highlights how the costs are divided: reads are cheaper, writes are more expensive, especially cold writes.

TABLE 2.2: Loan Gas consumption.

Function name	Min	Average	Median	Max
repay	132346	162272	163834	209640
is_failed	21540	23458	23650	23650
is_expired	21430	21430	21430	21430

`repay` is by far the most expensive operation at 162000 gas, but little of that is `repay` itself: the reporter only calculates external function calls. The cost sits in `distribute_base` and `distribute_interest`, each performing a payment and an overall increasing cost based on the number of contributors. At the other end we find `is_expired`, of which 21000 is the base transaction fee: it compares block.number against two `immutable` fields held in bytecode rather than storage.

TABLE 2.3: LoanService Gas consumption.

Function name	Min	Average	Median	Max
resolve_proposal	63294	981881	1235906	1337857
submit_proposal	189584	204243	206684	206708
terminate	75413	136047	136047	196681
set_migration_source	46176	46176	46176	46176
set_oracle	33202	33202	33202	33202
accept_admin	28262	28262	28262	28262
compensation_pool	23438	23438	23438	23438

`resolve_proposal` spans 68000 to 982000 gas, and the gap is almost entirely one thing: a rejected proposal returns before any Loan exists, while an approved one deploys a dedicated Loan contract. There is also the cost of sorting and locking each contributor's position which grows with the number of contributors. By contrast `accept_admin` costs 28262. After the 21000 base we can conclude that the real cost of the function is around 7k gas.

2.2 Reentrancy Protection

The project includes both a secure implementation (`LoanService.sol`) and an intentionally vulnerable version (`LoanServiceVulnerable.sol`) used to demonstrate a reentrancy attack. The vulnerability affects the `claim_compensation()` function, where the vulnerable contract transfers Ether before updating its internal state. A malicious contract can exploit this behavior by re-entering the function from its `receive()` callback while the previous execution is still in progress, allowing multiple compensation claims before the contributor's accounting information is updated. The secure implementation follows the Checks–Effects–Interactions pattern by updating the contract state before performing external transfers, preventing recursive calls from reusing outdated accounting information. The attack is implemented in `ReentrancyAttacker.sol` and validated through dedicated Hardhat tests, which confirm that the vulnerable contract can be exploited while the protected implementation resists the same attack.

2.3 Discussion of Malicious Strategies

When a loan is partially repaid, base is refunded to contributors in order of locked value, highest first. A partial repayment therefore fully refunds the largest lockholders before smaller ones receive anything. If the loan is only partially repaid and then fails, the contributors at the top of the list are made whole by real repayment, while those at the bottom are left owed and must rely on the compensation pool, which may be empty or insufficient.
A contributor deliberately makes their locked value in a loan the largest, so they sit at the front of the refund order. Because locking is proportional to disposable value at resolution time, the attacker can do this by timing a large deposit just before `resolve_proposal` is called on a proposal. When that loan is created, the attacker gets the biggest proportional lock, hence refund priority. If the applicant makes any partial repayment before the loan expires, the attacker will receive most of the base repayment while the honest smaller contributors get very little from the applicant and are pushed onto the much smaller compensation pool. Over many loans, the attacker systematically collects real repayments while honest contributors receive less than what they would have been repaid in a normal scenario. The attacker can then withdraw once the loan resolves, having been exposed to risk for only a short window.

Chapter 3

User Manual

3.1 Requirements

The project was developed and tested using **Geth** version `1.13.15-stable-c5ba367e`, **Node.js** version `v22`, **Hardhat** version `v3.9.0`, **Solidity** version `v0.8.28`, and **Apache Maven** version `v3.9.9`.
The Bitcoin blockchain blocks required are `blk00000`, `blk00001`, `blk00002` and must be manually copied into the `bt_chain_blocks` directory together with the `xor.dat` file.

3.2 Project Structure

The **bt_chain_blocks/** directory contains the local Bitcoin blockchain blocks, while the **chain/** directory stores the local Ethereum configuration, including the genesis file, accounts, and Geth data. The **hardhat/** directory contains the Solidity smart contracts and tests. The **offchain-scanner/** module includes the Java application responsible for extracting UTXO data from the Bitcoin blockchain. The **python/** directory provides scripts for deployment, oracle management, auto voting, and system demo, while the **state/** directory stores the deployment state shared by the Python scripts. The **tool.py** script is an orchestration tool for managing the entire system, and the **venv/** directory contains the Python virtual environment and dependencies.

3.3 Orchestration Script

The `tool.py` script provides an interactive command line interface to automate the setup and execution of the system. The operations are to be executed following the order of the list:

1. **Hardhat build.**
2. **Run the Off-chain Scanner.**
3. **Setup the Python virtual environment.**
4. **Initialize the local Ethereum blockchain.**
5. **Start the local Geth node** (keep alive).
6. **Deploy the application.**
7. **Start the Oracle Daemon** (keep alive).
8. **Start the Auto Voter** (keep alive).
9. **Run the demonstration** (keep alive).
10. **Attach to Geth.**